Fiche de Discours · Présentation Oral S4

Projet **SkillSwap** · Validation 1 (Conception & Architecture)

Amal Jemli
Responsable : *Skill Service (MS2)*
ESPRIT · 4DAS6 - Groupe 5

1. PARTIE COMMUNE (INTRODUCTION GÉNÉRALE)

🖥️ Slide Commune 1 : Technologies choisies

⏱️ 15 - 20 sec

> « Pour nos technologies :
> • Nous utilisons **Spring Boot 3** avec **Java 17** pour le backend, sécurisé par **Keycloak** et routé par **Spring Cloud Gateway**.
> • La communication est assurée par **OpenFeign** en synchrone et **RabbitMQ** en asynchrone.
> • Chaque microservice possède sa propre base de données, et l'ensemble de nos APIs est documenté sous **Swagger**. »

👉 *Conseil : Balayez calmement l'écran du regard, montrez la cohérence de la stack technique.*

🖥️ Slide Commune 2 : Découpage en microservices et bases de données

⏱️ 20 - 25 sec

> « Notre architecture est découpée en 7 microservices indépendants avec le principe *Database per Service* :
> • Nous avons les services pour les utilisateurs, les échanges, les crédits, les avis et les notifications.
> • Moi, je suis responsable du **Skill Service** sur le port **8082** avec **MySQL**, qui gère le catalogue et les compétences des membres.
> • Et nous avons un service de recommandation commun avec **Neo4j** pour le matching. »

👉 *Conseil : Pointez du doigt votre ligne **MS2** dans le tableau en disant « c'est ma partie, que je vais vous détailler ».*

2. PARTIE INDIVIDUELLE (SKILL SERVICE)

🖥️ Slide 1 : Description et responsabilités

⏱️ 10 sec

> « Bonjour. Je suis Amal, responsable du **Skill Service**.
> Mon rôle est de gérer le catalogue des compétences de SkillSwap, et de permettre à chaque membre de déclarer ce qu'il offre ou ce qu'il veut apprendre. »

🖥️ Slide 2 : Diagramme de classes

⏱️ 15 sec

> « Sur mon modèle, j'ai trois entités : **Category**, **Skill**, et l'entité pivot **UserSkill**.
> Un membre peut choisir son niveau et le type : **OFFERED** *(ce qu'il propose)* ou **WANTED** *(ce qu'il recherche)*.
> **Point clé :** le `userId` est un simple identifiant sans clé étrangère SQL, pour respecter la règle *Database per Service* avec ma base MySQL indépendante. »

👉 *Conseil : Pointez la classe `UserSkill` et insistez sur l'absence de clé étrangère SQL.*

🖥️ Slide 3 : Principaux endpoints REST

⏱️ 15 sec

« Pour mes APIs REST :
• La lecture du catalogue est **publique** pour tout le monde.
• Mais pour ajouter une compétence, il faut être connecté avec un **jeton Keycloak**.
J'ai déjà testé et validé mon API `GET /api/skills` sur **Swagger UI**, et elle fonctionne parfaitement avec un code **200 OK**. »

👉 *Conseil : Mentionnez que le test Swagger UI est documenté dans le contrôleur sur GitHub.*

🖥️ Slide 4 : Événements RabbitMQ

⏱️ 15 sec

« Sur cette slide, voici mes événements asynchrones avec **RabbitMQ** :
• Je **publie** `userskill.created` pour que le service de recommandation mette à jour son matching.
• Et je **consomme** `user.deleted` pour désactiver automatiquement les compétences si un utilisateur supprime son compte.
Chaque événement contient un `eventId` unique pour éviter les doublons. »

🖥️ Slide 5 : Carte globale des interactions

⏱️ 12 sec

« Ici, sur l'architecture globale, mon service est **100% autonome**.
Comme vous le voyez sur la note : il n'a **aucun appel synchrone sortant**. Il répond aux autres services, mais il n'appelle personne en direct.
Donc même si un autre service tombe en panne, le Skill Service continue de fonctionner. »

👉 *Conseil : Montrez la note jaune en bas à gauche de la slide. C'est votre grand point fort !*

🖥️ Slide 6 : Séquence synchrone OpenFeign

⏱️ 15 sec

« En synchrone : l'**Exchange Service** m'appelle avec **OpenFeign** pour vérifier une compétence avant de valider un échange.
• Si la compétence existe : je réponds **200 OK**.
• Si elle n'existe pas : je réponds **404**.
• Et si mon service est en panne : le disjoncteur **Resilience4j** déclenche un repli **503** pour protéger l'échange. »

🖥️ Slide 7 : Séquence asynchrone RabbitMQ

⏱️ 15 sec

« Pour finir, ce schéma montre le flux asynchrone complet :
La communication passe par le topic `skillswap.events` avec des files durables, pour qu'aucun message ne soit perdu.
Tout mon code et ma documentation sont déjà commités et pushés sur ma branche GitHub `skill-service`. Je vous remercie ! »

3. RÉPONSES RAPIDES AUX QUESTIONS FRÉQUENTES DU JURY

❓ Pourquoi userId est un simple Long sans clé étrangère SQL ?

👉 **Réponse :** « C'est la règle d'or *Database per Service* des microservices. Le Skill Service a sa propre base MySQL (`skill_db`) et ne doit jamais dépendre de la base du User Service. Le `userId` est extrait directement du token JWT Keycloak. »

❓ Pourquoi n'avez-vous aucun appel synchrone sortant ?

👉 **Réponse :** « Pour garantir une autonomie totale et une résilience maximale. Si un autre service tombe en panne, le Skill Service ne subit aucun effet domino et continue de répondre à 100%. »

❓ Avez-vous testé votre service ?

👉 **Réponse :** « Oui, le microservice démarre sur le port 8082, communique avec Eureka, et l'API `GET /api/skills` a été testée et validée avec succès sur Swagger UI avec un code 200 OK. C'est mentionné dans mon contrôleur et pushé sur ma branche GitHub. »